

Efficient Range Deletes in LSM Engines

Yu-Cheng Huang
Boston University
USA

Boston University
USA

Boston University
USA

ABSTRACT

In contemporary times, the LSM-Tree is widely adopted for its efficient insertion capabilities. However, a drawback arises when dealing with range deletes, as they are stored in individual SST files. Consequently, the process of searching for a deleted key necessitates opening and reading a file, leading to wasteful disk IO operations and time consumption.

This paper focuses on addressing this issue by proposing a range delete filter (RDF) to minimize disk IOs during the search for deleted keys. The RDF serves as a mechanism to swiftly identify and handle deleted keys, optimizing the overall performance of the LSM-Tree.

ACM Reference Format:

Yu-Cheng Huang, ****, and ****. 2023. Efficient Range Deletes in LSM Engines. In *Proceedings of ACM Conference (Conference'17)*. ACM, New York, NY, USA, 10 pages. <https://doi.org/10.1145/nnnnnnn.nnnnnnn>

1 INTRODUCTION

LSM-based Key-Value Stores. With the arrival and development of 5G, AGI, XR, more and more data are generated and stored on the servers. A bunch of database system adopt LSM as its underlying storing structure for its writing efficiency. Data first write directly into buffer until the buffer is full, then the whole data block is flushed on to the disk as an immutable SSTable with keys sorted. There're several levels in which their size grows larger with ratio T. Once the level reaches its maximum capacity, several SSTable files are gathered and merged-sorted before moving to the next level.

Range Delete in LSM Trees. Typically, range deletes are stored as an range delete tombstones with start and end keys indicating their coverage inside the SSTable files where the file ranges overlapped with the key ranges. $\neg$ In RocksDB, when compaction happens, these range deletes purge out the obsolete entries across all the compacted files.

Problems. We may waste time searching for deleted items on the disk because range deletes are stored in the SSTable files. To know whether an entry exist, there's a chance we have to open and read the file if not blocked by fencing index

and bloom filter. However, there is a huge speed gap between fetching data from disk and from the memory, which ends up slowing our queries. ($\neg$ Assumption)

1.1 Motivation

If we can use range delete information to set an filter (RDF) inside the buffer to decrease numbers of IO access, then the speed of data query can be accelerated. ($\neg$ Assumption) With RDF, we can directly purge out queries on non-existed keys and prevent unnecessary disk IOs.

1.2 Problem Statement

Searching for deleted items on the disk can be time-consuming due to the storage of range deletes in SSTable files. Initially, metadata is read from the file to check for the existence of a specific key. If a key is deleted, a range deletion can be found in an exclusive area within the file, but this requires an additional disk I/O operation to read the page containing range deletes. However, if the range deletion is in the buffer, no extra I/O operations are needed to access the pages containing the range deletes.

Determining the presence of deleted items on the disk can lead to time wastage because range deletes are stored in SSTable files. The process involves first reading metadata from the file to ascertain whether a particular key exists in the file. To determine whether a key is deleted, we need to look for a range deletion in a specific area within the file, which requires an additional disk IO operation to read the page containing the range deletion information. However, if the range deletion is present in the buffer, no additional IO operations are required to access the pages containing the range deletes.

To determine the existence of an entry, there is a possibility of having to open and read the file unless it is blocked by a fencing index and bloom filter. Unfortunately, fetching data from disk is significantly slower than fetching from memory, resulting in a slowdown of query workload. ($\neg$ Assumption)

1.3 Contributions

- (a) discuss and analyze influence of different RDFs (PLRDF, split-PLRDF, toplevel-RDF, Skyline-RDF) and their memory footprint
- (b) get rid of the requirement of sequence number for every entry

2 LSM BACKGROUND

The LSM tree is a structure optimized for writes and consists primarily of two operations: Flush and Compaction. It

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

Conference'17, July 2017, Washington, DC, USA

© 2023 Association for Computing Machinery.

ACM ISBN 978-x-xxxx-xxxx-x/YY/MM. . . \$15.00

<https://doi.org/10.1145/nnnnnnn.nnnnnnn>

consists of multiple levels with T-ratio up-growing capacity. **Leveling and Segmented LSM.** Data is stored in SSTable files on each level, with non-overlapping ranges between files. During compaction, only selected files are moved to the next level.

Flush. When the buffer is full, data is sorted and written into an immutable sorted string table on disk.

Compaction. Triggered when a level is full, several files will be picked out following some rules and sort-merged with the overlapping files on the next level.

Write Amplification and Space Amplification. Space amplification occurs due to the presence of the same key on different levels. Write amplification happens when duplicate keys in the database result in additional writes when moving data to the next level.

Range Delete. Range deletes are stored as range tombstones, represented by a triple of start value, end value, and sequence number in each SSTable file that contains their ranges.

3 PROBLEM

In rocksDB, range deletes hold information about non-existent entries, but they are solely stored in files, resulting in additional disk IO operations in accessing them, inevitably declining query speed. Presently, the bloom filter only stores details about point entries, and fencing pointers store information about existing file ranges. However, there is no filter that stores delete ranges, making it impossible to determine whether a specific point is deleted without resorting to disk IO.

First, current system loss a degree of freedom for not keeping range delete information in the buffer. One advantage of rocksDB is that users can choose to keep the bloom filter inside the buffer, enabling them to filter out non-existent keys in a given file without needing direct disk access. Unfortunately, this option is not available for range deletes. When a user wants to determine if a key is deleted by a range delete or not, performing disk IO operations becomes necessary. Consequently, we lose the freedom to remove range-deleted keys before directly accessing the disk, leading to potential impacts on performance.

In Addition, without range delete filter, systems take more time in reading an SST file. For a system, files are read in blocks and each file contains multiple blocks. To read an SST file, rocksDB first reads the metadata to identify which block may contain the target key or have overlapping deleted ranges with it. Then, it proceeds to find the key in the block that may contain the key and retrieve range delete information in range delete block. Since range deletes are currently stored with files on each level, the system may perform extra disk IOs to pinpoint the file where either the searching key or range deletes in interest lie in to determine whether a key exists or deleted.

4 SOLUTION NAME

4.1 Solution Design

RD can be either stored on each level (PLRDF) or on the top level (TopLevel-RDF). When range deletes flush down to the disk, range deletes are merged and stored on to level 0 vector. Latter, when compaction happens, corresponding range deletes are moved and merged to the next level vector in PLRDF. While in TopLevel-RDF, range deletes are stored in the top level only, and those whose ranges intersects with the newly inserted files are removed. When either flush or compaction happens, range deletes from the top level which are newer remove the obsolete entries from the files it compacted to on the lower level. In this way, if an entry is overlapped with a range delete tombstone in a file, it must be newer than the range delete tombstone. Sequence numbers are not required because if there are overlapping files across two levels, files on the lower level is newer and if an entry and range deletes co-exist in a file, the entry is sure to exist.

- **None.** Original rocksDB without RDF, it performs similar to PLRDF, while it searches for overlapping range deletes first each time before searching a specific key in a file.
- **PLRDF.** Range deletes are stored on each level and moved down to the next level during compaction. When a point query is blocked on a certain level, it indicates that the key doesn't exist on the levels below, requiring disk access on the current level, just the same as storing range deletes inside SST files. It performs a key search in a file first, if the key is found it return the key directly before it try to search for the overlapped key in the same file.
- **Split-PLRDF.** Whenever new entries arrive at a certain level, ranges that overlap with the newly inserted entries are split. This ensures that an entry existing on a certain level won't be covered by ranges in the RDF. However, this approach results in a larger memory footprint due to fragmented ranges.
- **TopLevel-RDF.** RDF is limited to the top level, such that it blocks query if an entry doesn't exist in the database. It performs split function on all incoming entries, leading to larger memory footprint.
- **Skyline-RDF.** All the range deletes inserted are maintain in this single RDF. Keys are first found before comparing with this RDF. Because we need to deferentiate the whether a key with arbitrariness sequence number is deleted, each ranges within this RDF needs a sequence number too, so that it can tell whether a key is truly deleted. And due to the post filtering, it spent more effort than the previous RDFs, where if a range deleted was found on a certain level, originally it shall directly return but now it keeps searching for the key until a key is found or reaches the last level.

(a) PLRDF: Range deletes are stored on each level and moved down to the next level during compaction. When a point

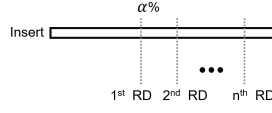


Figure 1: The first RD is inserted at the α of the total insert, and all the following RDs are equally separated between the first RD the end of the insert workload.

query is blocked on a certain level, it indicates that the key doesn't exist on the levels below, requiring disk access on the current level, just the same as storing range deletes inside SST files..

(b) Split-PLRDF: Whenever new entries arrive at a certain level, ranges that overlap with the newly inserted entries are split. This ensures that an entry existing on a certain level won't be covered by ranges in the RDF. However, this approach results in a larger memory footprint due to fragmented ranges.

(c) TopLevel-RDF: RDF is limited to the top level, such that it blocks query if an entry doesn't exist in the database. It performs split function on all incoming entries, leading to larger memory footprint.

5 EVALUATION

5.1 Experiment Settings

- cpu:
- ram:
- ssd:
- rocksdb:

workload settings:

Workload composed 2 parts, the first part is the insert workload, the second part is the query workload. Within the insertion workload, there're entry insert and RD while the RDs are equally distributed with an variable α that decides where the first RD starts at, which is shown in Figure .

In 1.

In the following part, read buffer is disabled. And Point Query is performed after all the insertions has been done.

From Figure 2 and Figure 3, one with Skyline and one without Skyline, we can see that the Skyline RDF is the worst of the all when performing PQ on the existing keys, because it doesn't return when the level contains RD, instead it only return when a key is found or going through all the levels. While the other RDF returns if they find a key is deleted by RD on a certain level. Comparing to Figure 5 which all the RDFs have similar work time, this is due to the fact that when performing PQ on existing keys, there's no chance of early return, which means that all doing the search until they reach the level where key lies.

As the Figure 4 shows, IOs occupies more than half of the time, if we can reduce the number of IOs, we can reduce the total query time.

$T = 2, P = 32, B = 4, E = 1024, \text{write_buffer} = 128.0 \text{ MB}$
 $\alpha = 0.9$

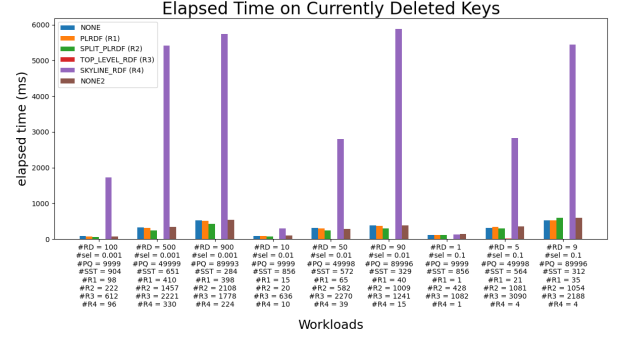


Figure 2: Elapsed time of testing on all of the deleted keys when $\alpha = 0.9, T = 2, P = 32, E = 1024$. From left to right separated into 3 groups [(g1,g2,g3), (g4,g5,g6), (g7,g8,g9)], each of them has selectivity 0.001, 0.01, 0.1 respectively. While (g1,g4,g7) has 10% of total keys deleted and the middle three (g2,g5,g8) have 50%, whereas the tallest ones (g3,g6,g9) have 90% keys deleted.

$T = 2, P = 32, B = 4, E = 1024, \text{write_buffer} = 128.0 \text{ MB}$
 $\alpha = 0.9$

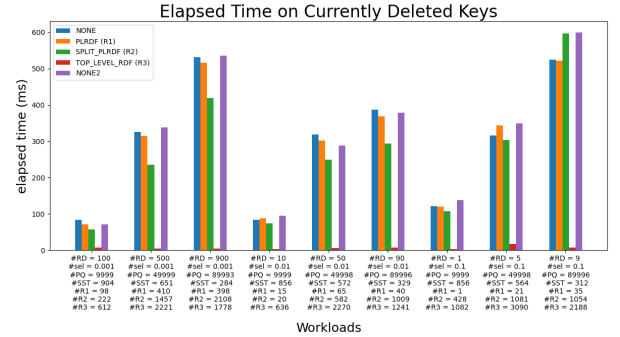


Figure 3: Elapsed time of testing on all of the deleted keys without Skyline when $\alpha = 0.9, T = 2, P = 32, E = 1024$. From left to right separated into 3 groups [(g1,g2,g3), (g4,g5,g6), (g7,g8,g9)], each of them has selectivity 0.001, 0.01, 0.1 respectively. While (g1,g4,g7) has 10% of total keys deleted and the middle three (g2,g5,g8) have 50%, whereas the tallest ones (g3,g6,g9) have 90% keys deleted.

Diving down to see the numbers of IOs spent on currently deleted keys in Figure 6 and Figure 7, there's no difference between PLRDF and default RocksDB implementation because when a file is opened the range deletes in a file are all read in to the buffer, when PQ is performed no extra IOs are spent to read in such meta data. Split-PLRDF and TopLevel RDF have less IOs consumption as expected.

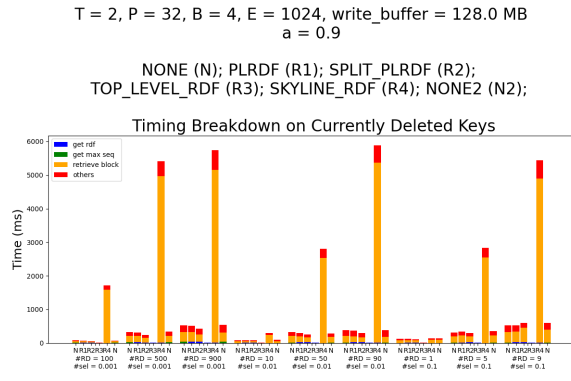


Figure 4: Timing breakdown of testing on all of the deleted keys when $\alpha = 0.9$, T = 2, P = 32, E = 1024. Over 9 workloads.

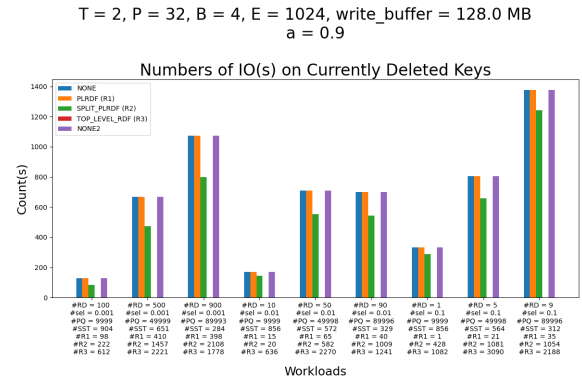


Figure 7: Elapsed time of testing on all of the existing keys when $\alpha = 0.9$, T = 2, P = 32, E = 1024. Over 9 workloads.

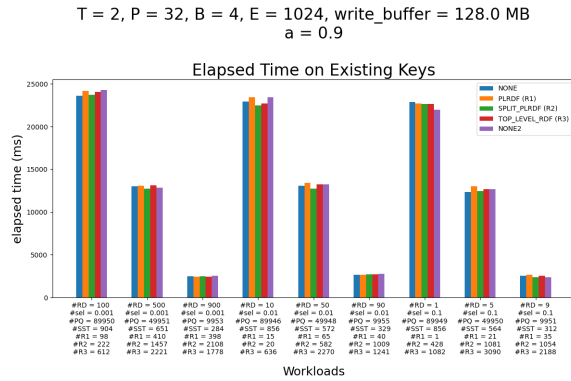


Figure 5: Elapsed time of testing on all of the existing keys when $\alpha = 0.9$, T = 2, P = 32, E = 1024. Over 9 workloads.

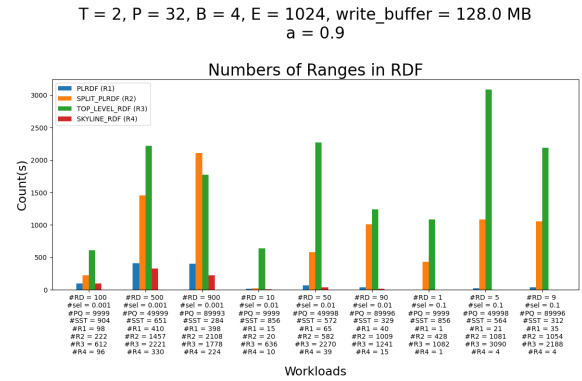


Figure 8: Number of ranges tested on all of the existing keys when $\alpha = 0.9$, T = 2, P = 32, E = 1024. Over 9 workloads.

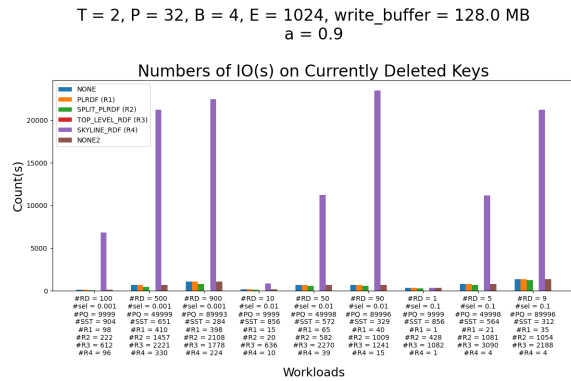


Figure 6: Elapsed time of testing on all of the existing keys when $\alpha = 0.9$, T = 2, P = 32, E = 1024. Over 9 workloads.

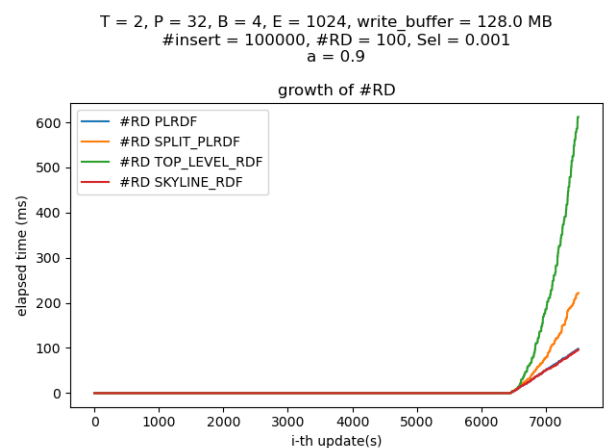


Figure 9: Numbers of ranges in RDs across time tested on all of the existing keys when $\alpha = 0.9$, T = 2, P = 32, E = 1024. Over 9 workloads.

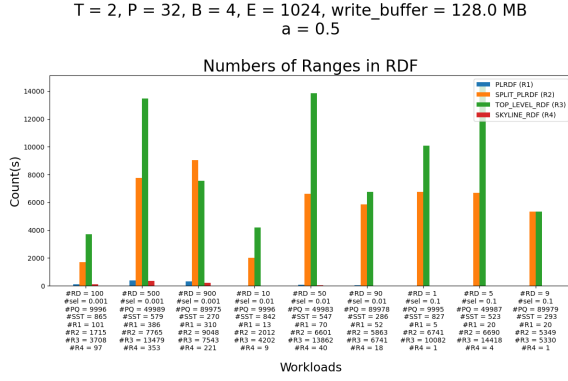


Figure 10: Number of ranges tested on all of the existing keys when $\alpha = 0.5$, T = 2, P = 32, E = 1024. Over 9 workloads.

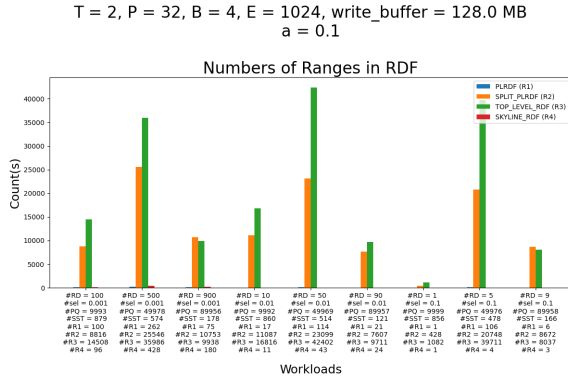


Figure 11: Number of ranges tested on all of the existing keys when $\alpha = 0.1$, T = 2, P = 32, E = 1024. Over 9 workloads.

When the times goes by, the number of ranges in RDs increases with inserted keys, as Figure 9 shows, the number of ranges in RDs increases almost linearly. The ones split by the inserted points grows faster, and the TopLevel is the worst of all because it stays on the first level and never discard any ranges, while the ranges can be merged if there's a new range overlapped with the old ones.

At the end of the insert, numbers of ranges in RDs are shown in Figure 8. Under most circumstances, TopLevel RDF is the worst for it keeps splitting and never discard any ranges. Skyline seems like the best while the PLRDF is almost comparable with the Skyline. When ranges are merged with a large input RD, numbers of ranges in TopLevel RDF drops temporarily, but it will soon increase again.

Now, when we decrease the value of alpha, making the first RD appeared earlier in the insertion workload, numbers of RDs increases, as in Figure 8, 10 and Figure 11 with α being 0.9, 0.5, 0.1 respectively. The numbers can be 10 times larger.

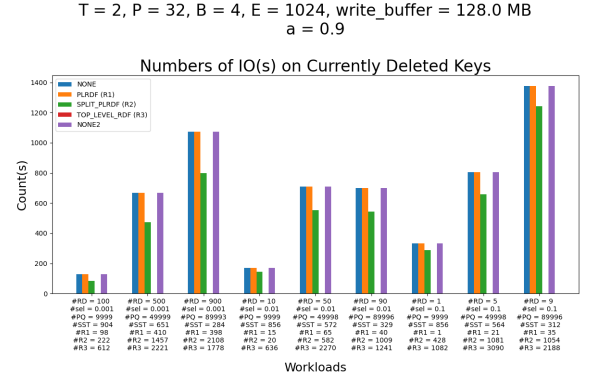


Figure 12: Number of IOs tested on all of the existing keys when $\alpha = 0.9$, T = 2, P = 32, B = 4, E = 1024. Over 9 workloads.

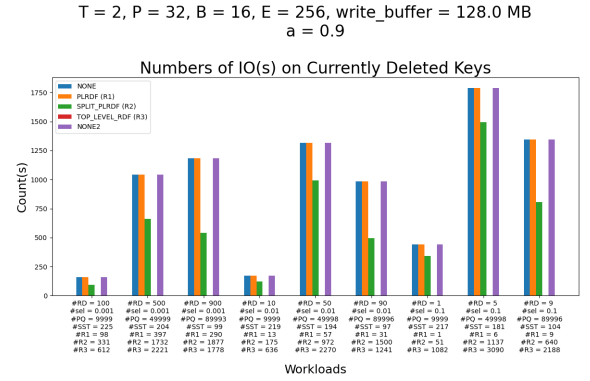


Figure 13: Number of IOs tested on all of the existing keys when $\alpha = 0.9$, T = 2, P = 32, B = 16, E = 256. Over 9 workloads.

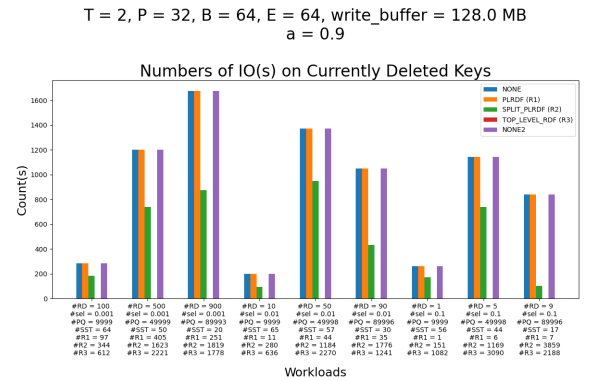


Figure 14: Number of IOs tested on all of the existing keys when $\alpha = 0.9$, T = 2, P = 32, B = 64, E = 64. Over 9 workloads.

Comparing Figure 12, 13 and 14, we can see that when the sizes of pages and write buffers are fixed and only varying on entry size, RDFs gain a small advantage when entry size becomes smaller, total numbers of IOs decrease slightly.

With Figure 15 and Figure 16, we see when E becomes smaller, the total size of the file becomes smaller, so is the trend of the total elapsed time. While the percentage of CPU

times comparing to Disk IOs time decreases too; This may cause by the fact that the retrieved data has to be processed.

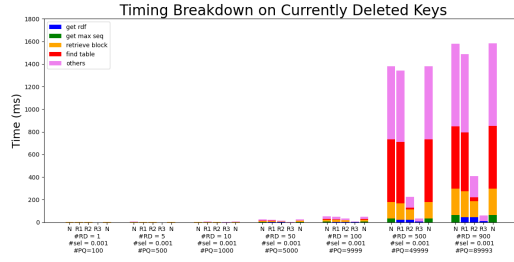
6 RELATED WORK

7 CONCLUSION

REFERENCES

T = 2, P = 512, B = 4, E = 1024, write_buffer = 2048.0 KB
a = 0.9

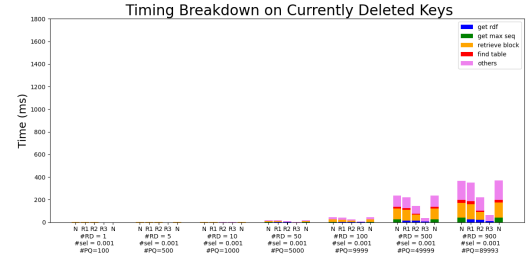
NONE (N); PLRDF (R1); SPLIT_PLRDF (R2); TOP_LEVEL_RDF (R3);



(a) Timing breakdown, tested on all of the existing keys when $\alpha = 0.9$, T = 2, P = 512, B = 4, E = 1024. Over 7 workloads.

T = 2, P = 512, B = 16, E = 256, write_buffer = 2048.0 KB
a = 0.9

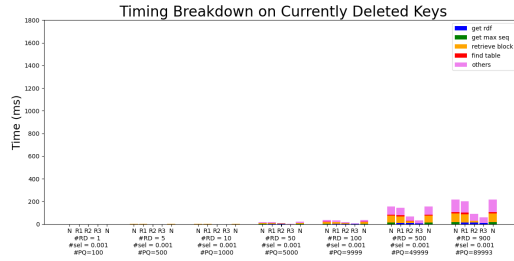
NONE (N); PLRDF (R1); SPLIT_PLRDF (R2); TOP_LEVEL_RDF (R3);



(b) Timing breakdown, tested on all of the existing keys when $\alpha = 0.9$, T = 2, P = 512, B = 16, E = 256. Over 7 workloads.

T = 2, P = 512, B = 64, E = 64, write_buffer = 2048.0 KB
a = 0.9

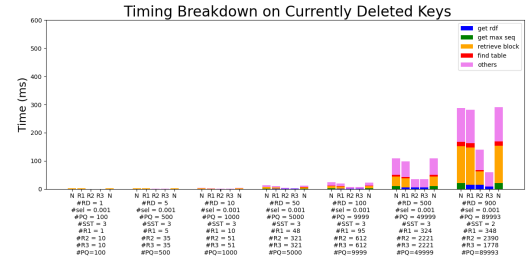
NONE (N); PLRDF (R1); SPLIT_PLRDF (R2); TOP_LEVEL_RDF (R3);



(c) Timing breakdown, tested on all of the existing keys when $\alpha = 0.9$, T = 2, P = 512, B = 64, E = 64. Over 7 workloads.

T = 2, P = 512, B = 256, E = 16, write_buffer = 2048.0 KB
a = 0.9

NONE (N); PLRDF (R1); SPLIT_PLRDF (R2); TOP_LEVEL_RDF (R3);

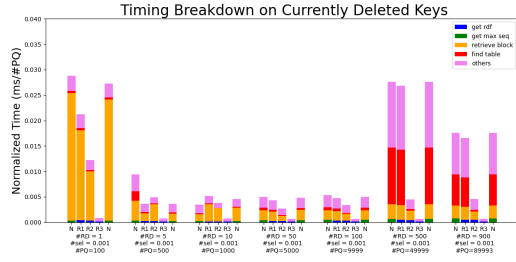


(d) Timing breakdown, tested on all of the existing keys when $\alpha = 0.9$, T = 2, P = 512, B = 256, E = 16. Over 7 workloads.

Figure 15: Normalized timing breakdown, varying on numbers of RDs for workload with $\alpha = 0.9$, T = 2, P = 512, B x E = 4096.

T = 2, P = 512, B = 4, E = 1024, write_buffer = 2048.0 KB
 $\alpha = 0.9$

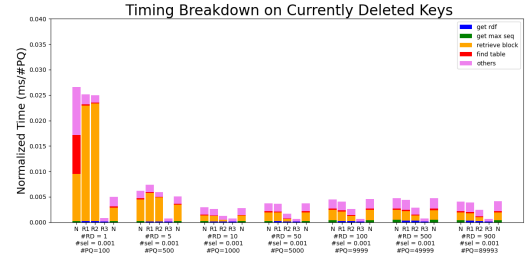
NONE (N); PLRDF (R1); SPLIT_PLRDF (R2); TOP_LEVEL_RDF (R3);



(a) Timing breakdown, tested on all of the existing keys when $\alpha = 0.9$, T = 2, P = 512, B = 4, E = 1024. Over 7 workloads.

T = 2, P = 512, B = 16, E = 256, write_buffer = 2048.0 KB
 $\alpha = 0.9$

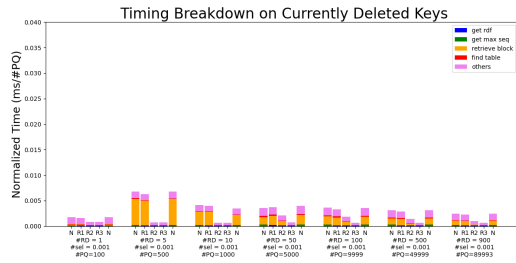
NONE (N); PLRDF (R1); SPLIT_PLRDF (R2); TOP_LEVEL_RDF (R3);



(b) Timing breakdown, tested on all of the existing keys when $\alpha = 0.9$, T = 2, P = 512, B = 16, E = 256. Over 7 workloads.

T = 2, P = 512, B = 64, E = 64, write_buffer = 2048.0 KB
 $\alpha = 0.9$

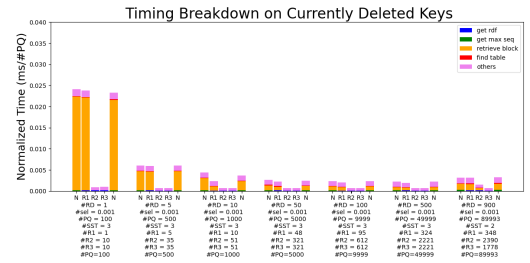
NONE (N); PLRDF (R1); SPLIT_PLRDF (R2); TOP_LEVEL_RDF (R3);



(c) Timing breakdown, tested on all of the existing keys when $\alpha = 0.9$, T = 2, P = 512, B = 64, E = 64. Over 7 workloads.

T = 2, P = 512, B = 256, E = 16, write_buffer = 2048.0 KB
 $\alpha = 0.9$

NONE (N); PLRDF (R1); SPLIT_PLRDF (R2); TOP_LEVEL_RDF (R3);

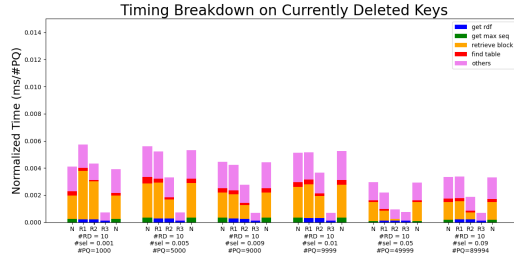


(d) Timing breakdown, tested on all of the existing keys when $\alpha = 0.9$, T = 2, P = 512, B = 256, E = 16. Over 7 workloads.

Figure 16: Normalized timing breakdown, varying on numbers of RDs for workload with $\alpha = 0.9$, T = 2, P = 512, BxE = 4096.

T = 2, P = 512, B = 4, E = 1024, write_buffer = 2048.0 KB
a = 0.9

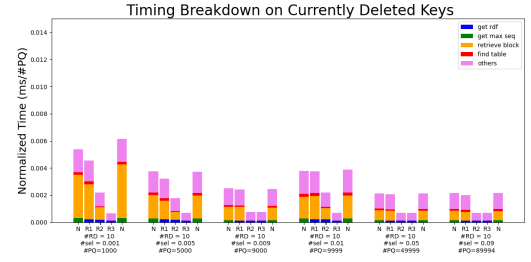
NONE (N); PLRDF (R1); SPLIT_PLRDF (R2); TOP_LEVEL_RDF (R3);



(a) Timing breakdown, tested on all of the existing keys when $\alpha = 0.9$, T = 2, P = 512, B = 4, E = 1024. Over 7 workloads.

T = 2, P = 512, B = 16, E = 256, write_buffer = 2048.0 KB
a = 0.9

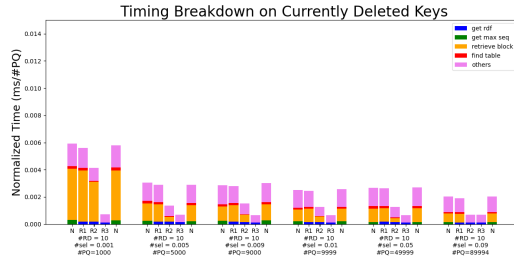
NONE (N); PLRDF (R1); SPLIT_PLRDF (R2); TOP_LEVEL_RDF (R3);



(b) Timing breakdown, tested on all of the existing keys when $\alpha = 0.9$, T = 2, P = 512, B = 16, E = 256. Over 7 workloads.

T = 2, P = 512, B = 64, E = 64, write_buffer = 2048.0 KB
a = 0.9

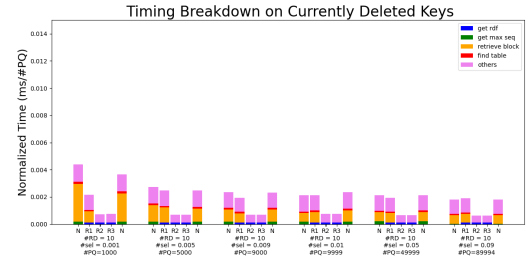
NONE (N); PLRDF (R1); SPLIT_PLRDF (R2); TOP_LEVEL_RDF (R3);



(c) Timing breakdown, tested on all of the existing keys when $\alpha = 0.9$, T = 2, P = 512, B = 64, E = 64. Over 7 workloads.

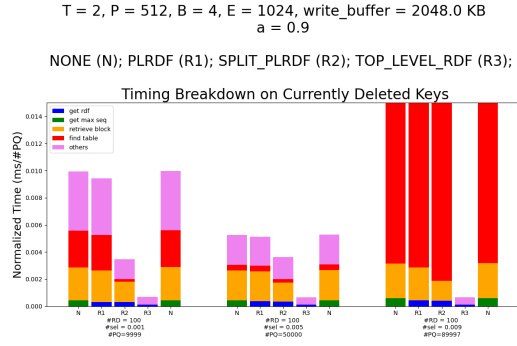
T = 2, P = 512, B = 256, E = 16, write_buffer = 2048.0 KB
a = 0.9

NONE (N); PLRDF (R1); SPLIT_PLRDF (R2); TOP_LEVEL_RDF (R3);

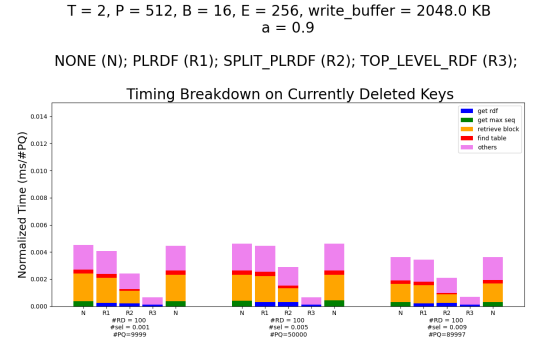


(d) Timing breakdown, tested on all of the existing keys when $\alpha = 0.9$, T = 2, P = 512, B = 256, E = 16. Over 7 workloads.

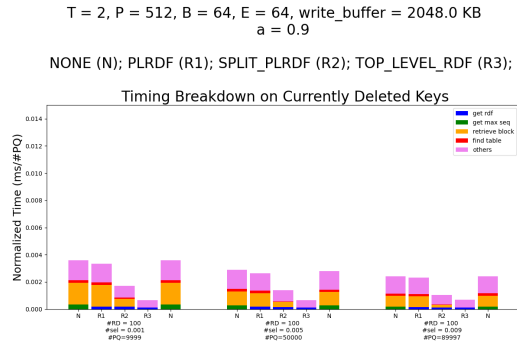
Figure 17: Normalized timing breakdown, varying on selectivity under numbers of RDs = 10 for workload with $\alpha = 0.9$, T = 2, P = 512, BxE = 4096.



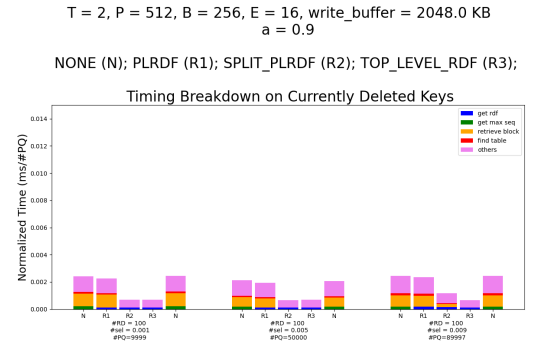
(a) Timing breakdown, tested on all of the existing keys when $\alpha = 0.9$, T = 2, P = 512, B = 4, E = 1024. Over 7 workloads.



(b) Timing breakdown, tested on all of the existing keys when $\alpha = 0.9$, T = 2, P = 512, B = 16, E = 256. Over 7 workloads.



(c) Timing breakdown, tested on all of the existing keys when $\alpha = 0.9$, T = 2, P = 512, B = 64, E = 64. Over 7 workloads.



(d) Timing breakdown, tested on all of the existing keys when $\alpha = 0.9$, T = 2, P = 512, B = 256, E = 16. Over 7 workloads.

Figure 18: Normalized timing breakdown, varying on selectivity under numbers of RDs = 100 for workload with $\alpha = 0.9$, T = 2, P = 512, Bx E = 4096.